

In [193... *#Let's import neccessary libraries from python*

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

In [196... *#Load CSV into enviornment*

```
df1 = pd.read_csv(r'C:\Users\parit\Downloads\AirPassenger.csv')
```

In [192... *#check on loaded data*

```
df1.head()
```

Out[192... **Year-Month Pax**

	Year-Month	Pax
0	1949-Jan	112
1	1949-Feb	118
2	1949-Mar	132
3	1949-Apr	129
4	1949-May	121

In [58]: df1.describe()

Out[58]: **Pax**

	Pax
count	144.000000
mean	280.298611
std	119.966317
min	104.000000
25%	180.000000
50%	265.500000
75%	360.500000
max	622.000000

In [59]: *#Remove index column and sent first column as index*

```
df1 = pd.read_csv(r'C:\Users\parit\Downloads\AirPassenger.csv', parse_dates = ['Yea
```

C:\Users\parit\AppData\Local\Temp\ipykernel_26468\2559609291.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df1 = pd.read_csv(r'C:\Users\parit\Downloads\AirPassenger.csv', parse_dates = ['Year-Month'])
```

```
In [60]: df1 = pd.read_csv(r'C:\Users\parit\Downloads\AirPassenger.csv', parse_dates = ['Year-Month'])
```

C:\Users\parit\AppData\Local\Temp\ipykernel_26468\1613511794.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

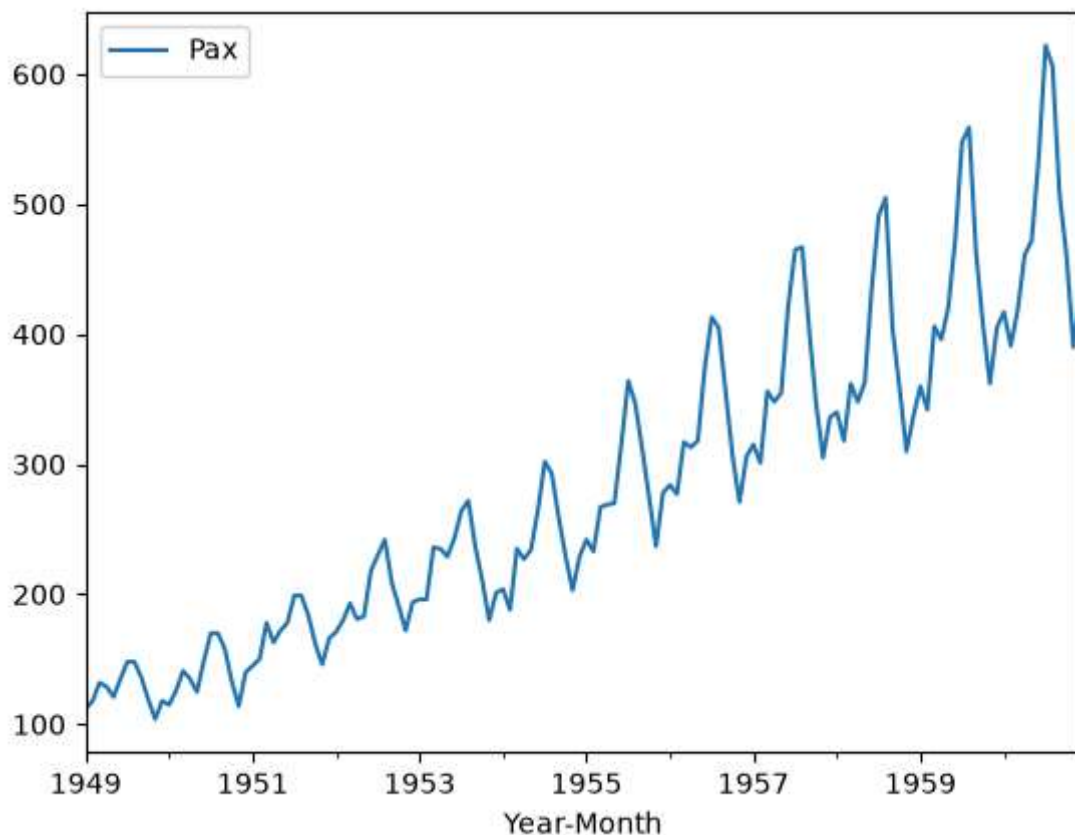
```
df1 = pd.read_csv(r'C:\Users\parit\Downloads\AirPassenger.csv', parse_dates = ['Year-Month'], index_col = ['Year-Month'])
```

```
In [61]: df1.head()
```

```
Out[61]:
```

	Pax
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

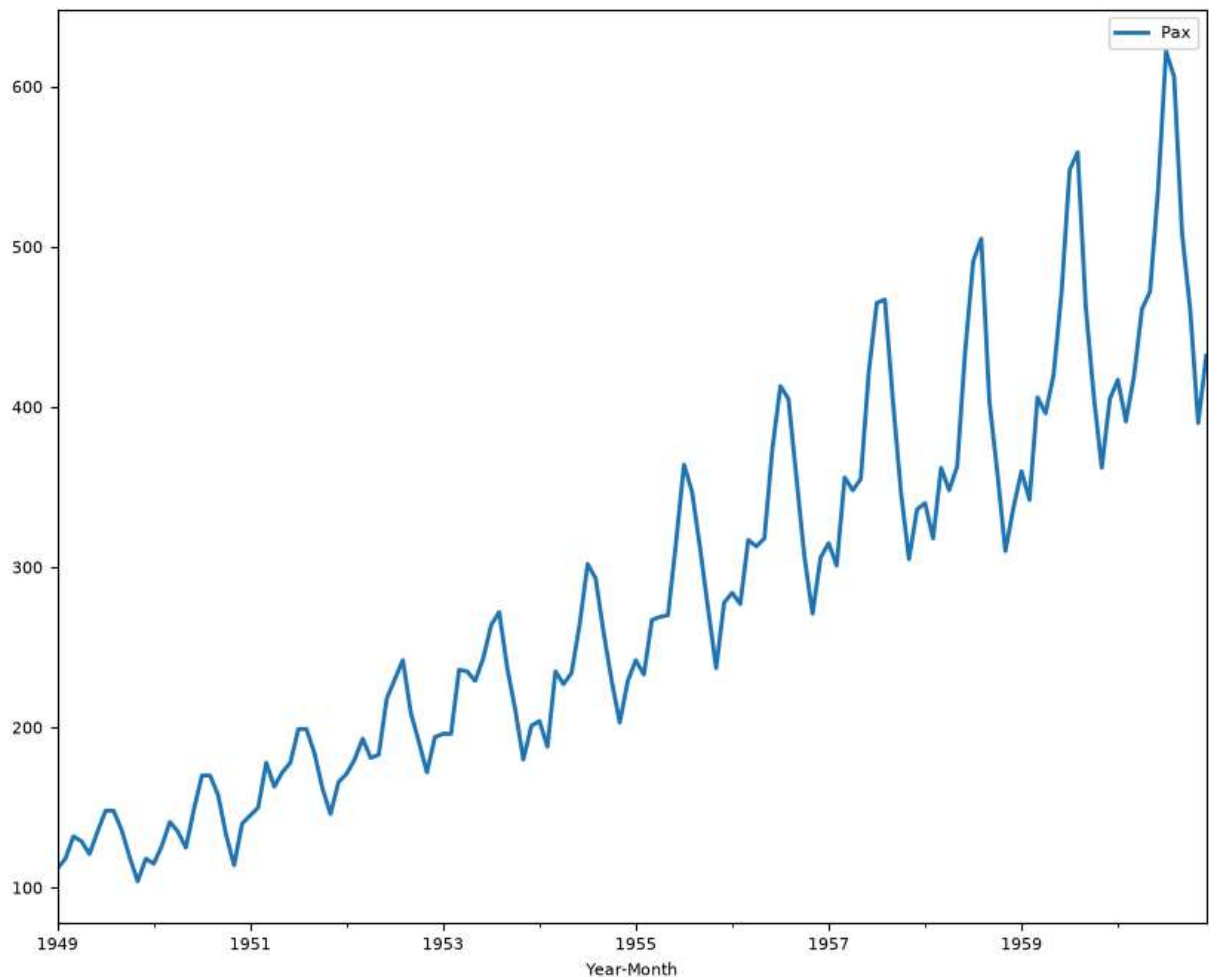
```
In [20]: df1.plot()
plt.show()
```



```
In [62]: #Control the plot
from pylab import rcParams
```

```
In [63]: rcParams['figure.figsize'] = 10,8
plt.rcParams['lines.linewidth'] = 2
plt.rcParams['font.size'] = 8

df1.plot()
plt.show()
```



```
In [64]: #check minimum and maximum in index for split.
```

```
print(df1.index.min())
print(df1.index.max())

df1.describe()df1.plot()

plt.title('Passenger over the years')
plt.xlabel('Year')
plt.ylabel('No. of Passenger')

plt.show()
```

```
1949-01-01 00:00:00
```

```
1960-12-01 00:00:00
```

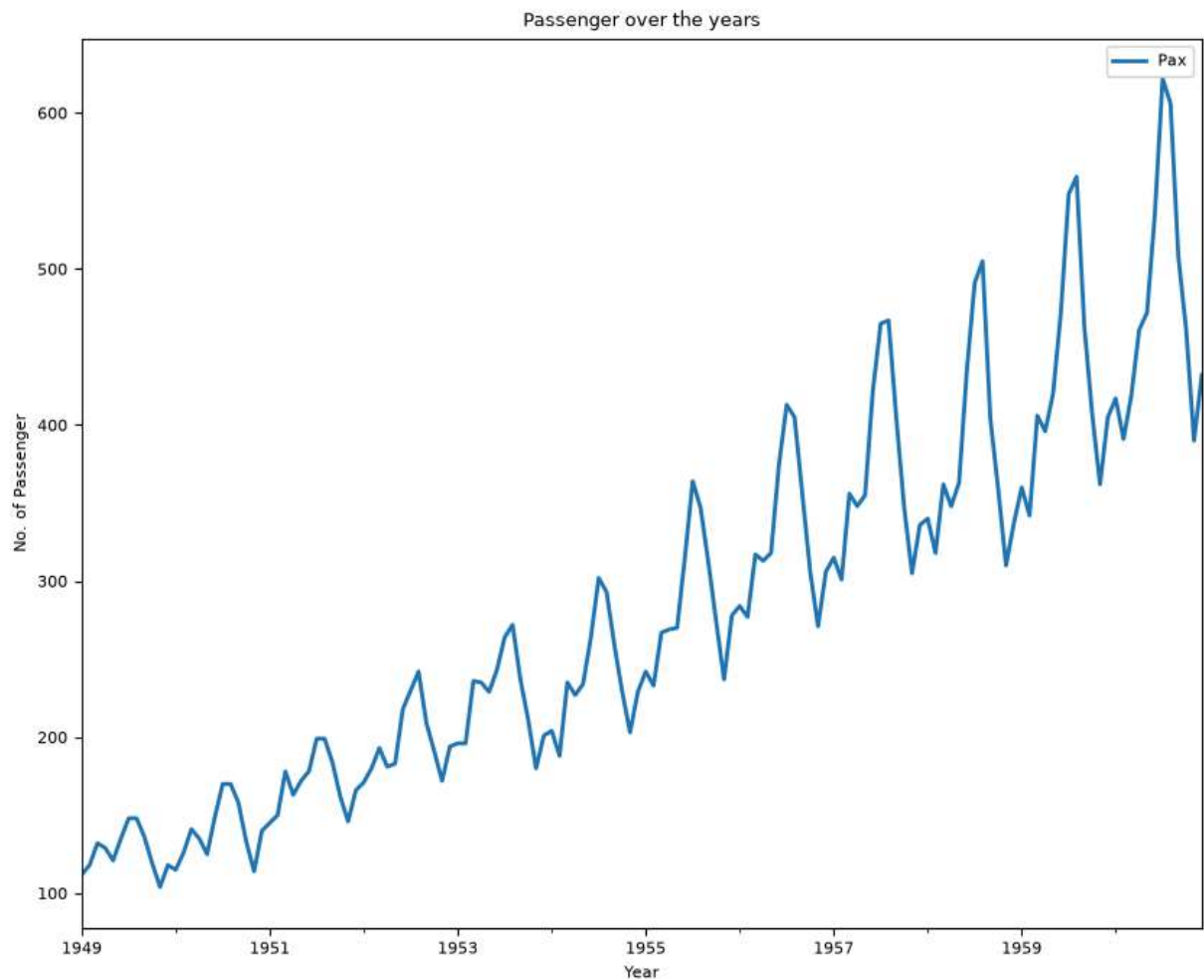
Out[64]:

	Pax
count	144.000000
mean	280.298611
std	119.966317
min	104.000000
25%	180.000000
50%	265.500000
75%	360.500000
max	622.000000

```
In [74]: df1.plot()

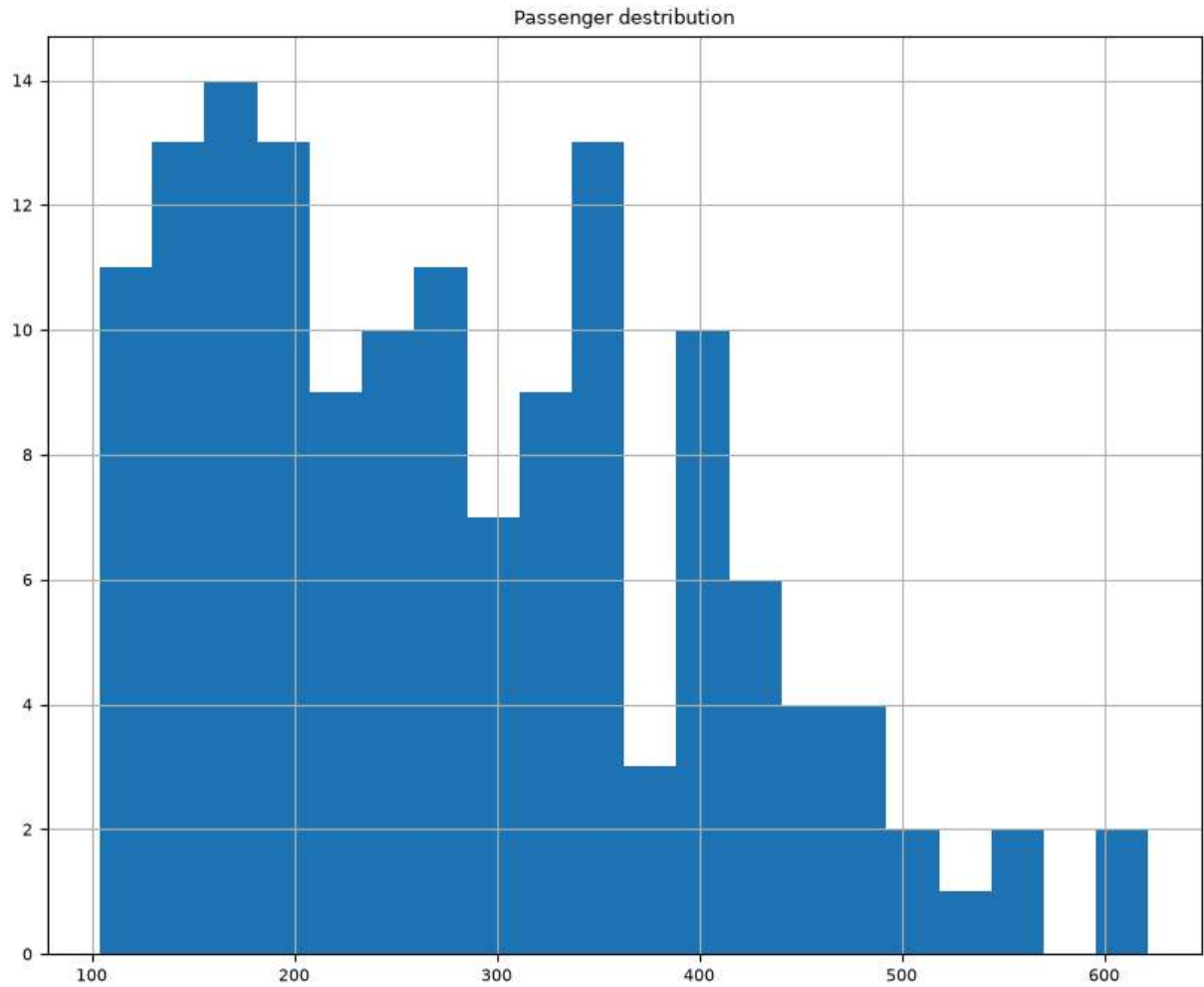
plt.title('Passenger over the years')
plt.xlabel('Year')
plt.ylabel('No. of Passenger')

plt.show()
```

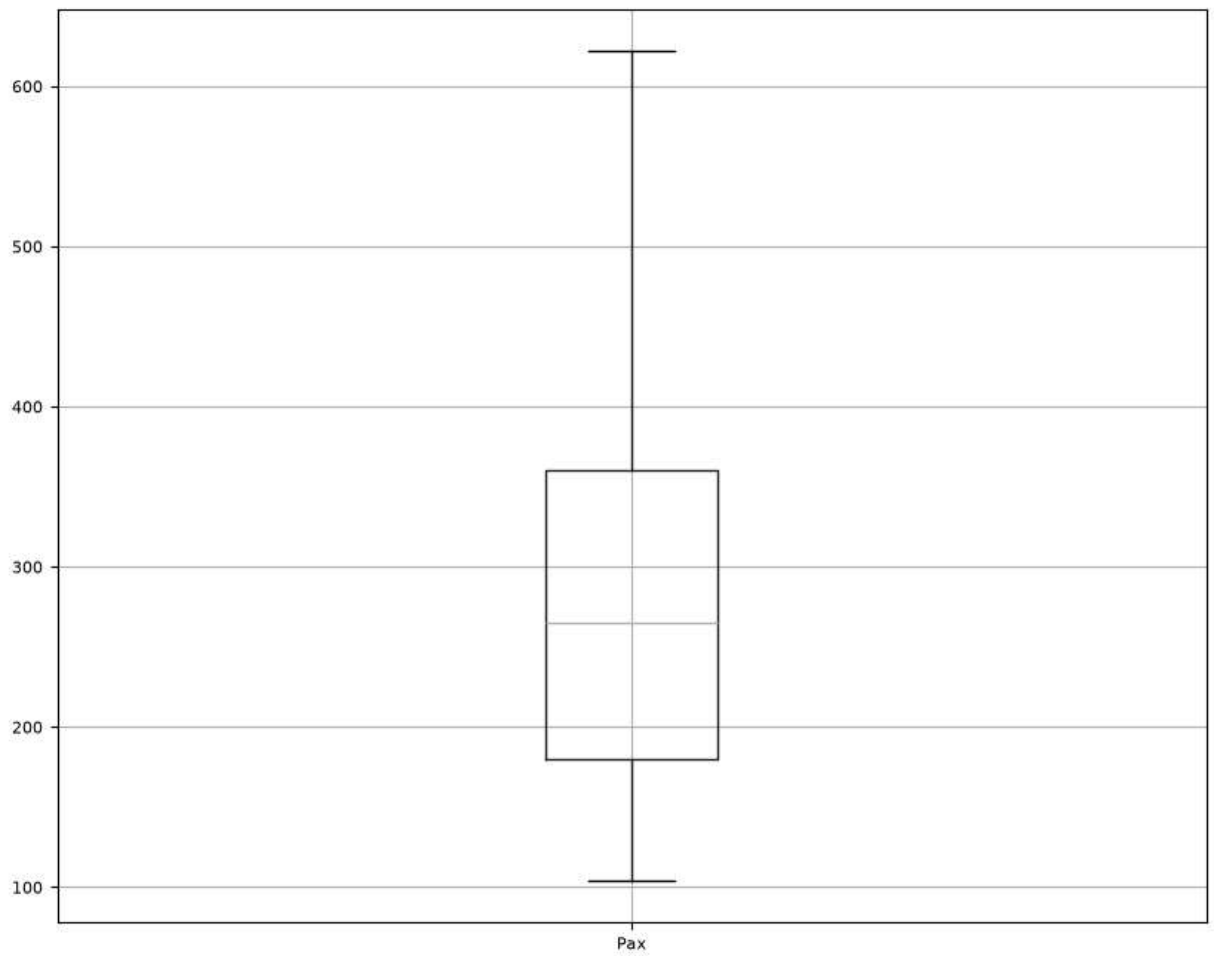


```
In [75]: ##Perform EDA
```

```
In [98]: df1['Pax'].hist(bins=20)  
plt.title('Passenger distribution')  
  
plt.show()
```



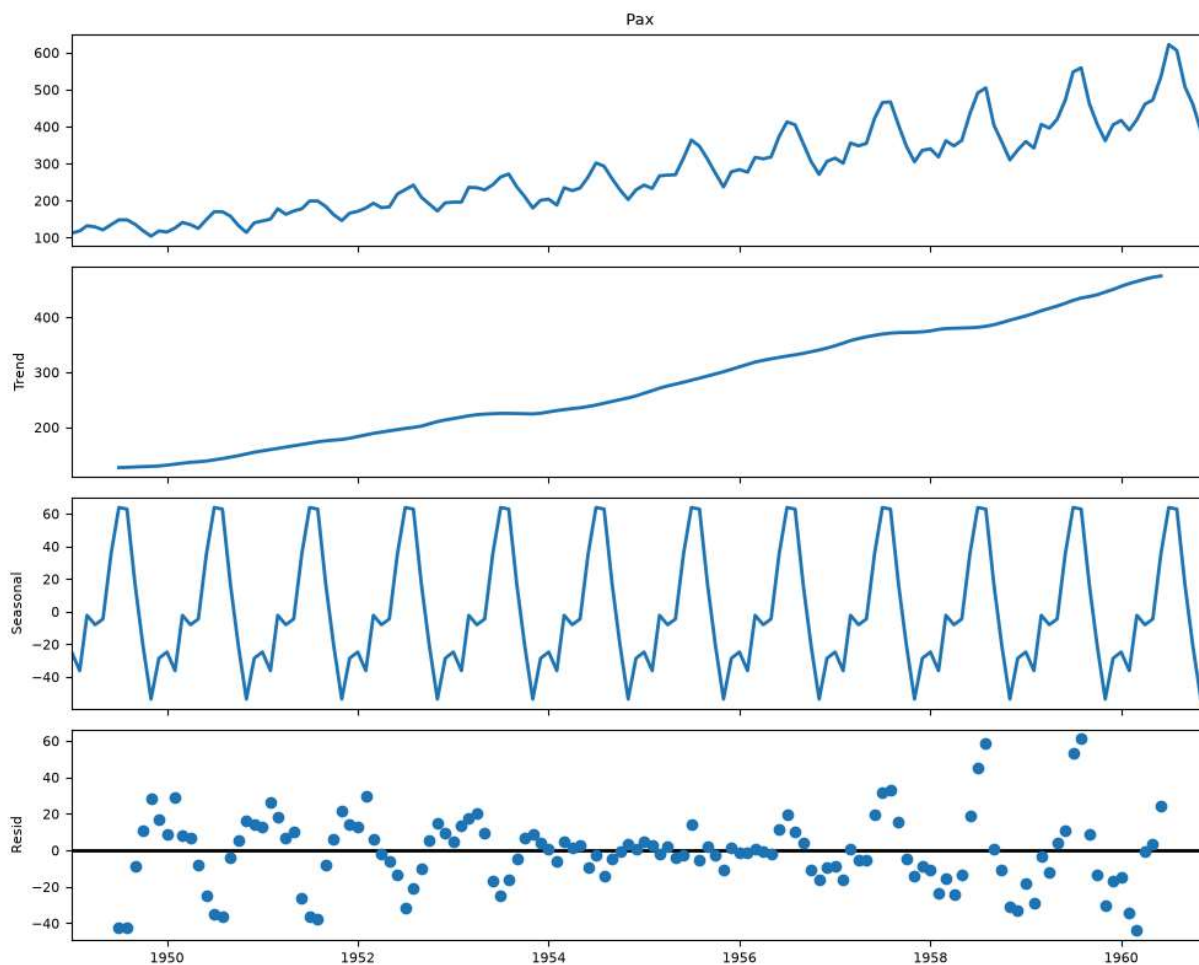
```
In [82]: df1.boxplot(column = 'Pax')  
plt.show()
```



```
In [83]: ##decompose data to gain - Trend, Seasonality and errors
```

```
In [86]: from statsmodels.tsa.seasonal import seasonal_decompose
```

```
In [91]: decom = seasonal_decompose(  
            df1['Pax'],  
            model = 'Multiplicative',  
            period = 12  
        )  
  
        decom.plot()  
        plt.show()
```



In [101...

```
## Log transformation to fix the large residual values

df1_log = pd.DataFrame()

df1_log['Pax'] = np.log(df1['Pax'])

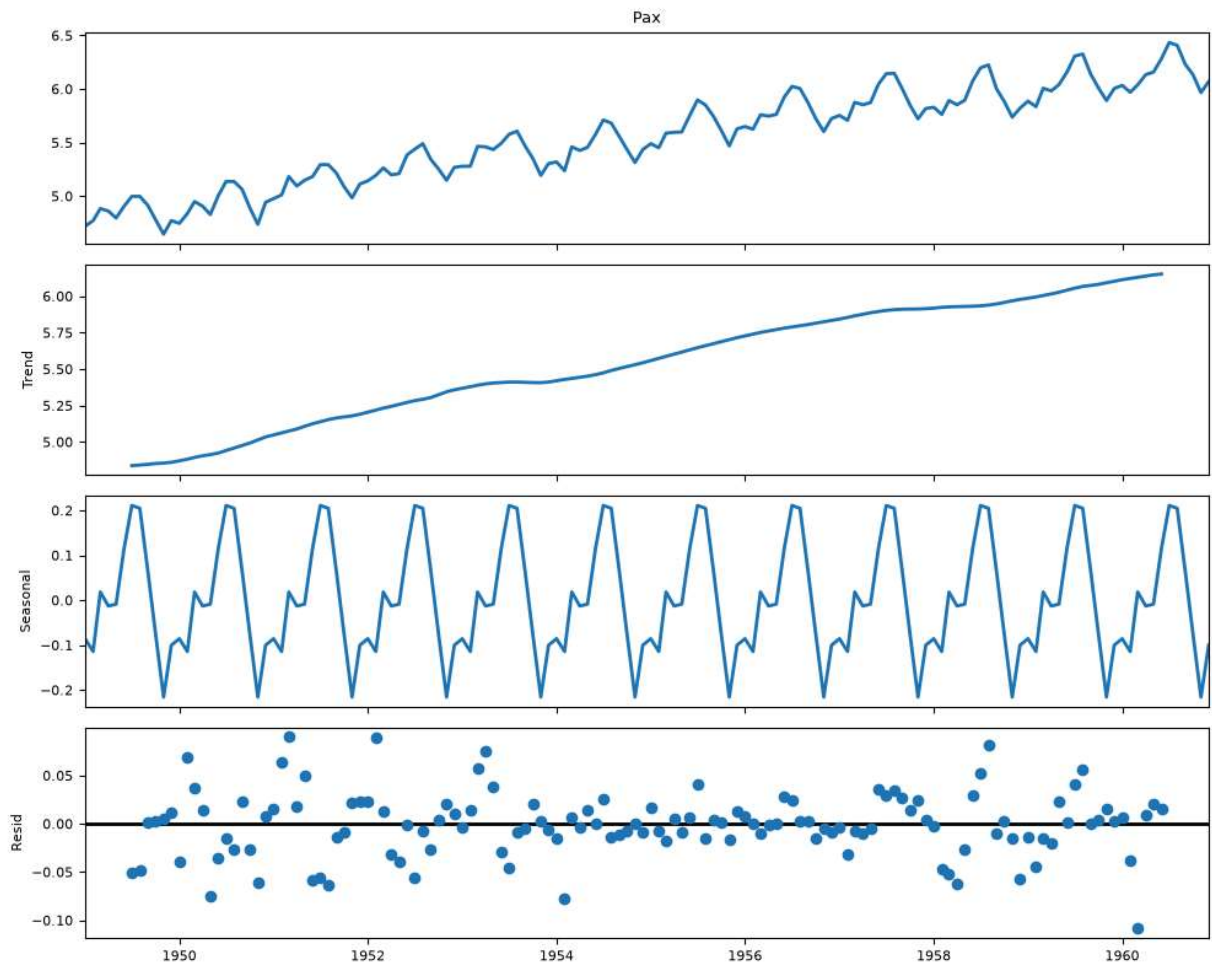
from statsmodels.tsa.seasonal import seasonal_decompose

df1_log = np.log(df1['Pax'])

result_log = seasonal_decompose(
    df1_log,
    model='additive',
    period=12
)

result_log.plot()
plt.show()

#Split the data into training and test. The test data should start from 1956.
```



```
In [94]: print(df1.index.min())
print(df1.index.max())
```

1949-01-01 00:00:00

1960-12-01 00:00:00

```
In [96]: train = df1[df1.index < '1956-12-01']
test = df1[df1.index >= '1956-12-01']
```

```
In [97]: print("Trainig Data")
display(train)
print("Test Data")
display(test)
```

Trainig Data

Pax	
Year-Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121
...	...
1956-07-01	413
1956-08-01	405
1956-09-01	355
1956-10-01	306
1956-11-01	271

95 rows × 1 columns

Test Data

Pax	
Year-Month	
1956-12-01	306
1957-01-01	315
1957-02-01	301
1957-03-01	356
1957-04-01	348
1957-05-01	355
1957-06-01	422
1957-07-01	465
1957-08-01	467
1957-09-01	404
1957-10-01	347
1957-11-01	305
1957-12-01	336
1958-01-01	340
1958-02-01	318
1958-03-01	362
1958-04-01	348
1958-05-01	363
1958-06-01	435
1958-07-01	491
1958-08-01	505
1958-09-01	404
1958-10-01	359
1958-11-01	310
1958-12-01	337
1959-01-01	360
1959-02-01	342
1959-03-01	406
1959-04-01	396

Year-Month	Pax
1959-05-01	420
1959-06-01	472
1959-07-01	548
1959-08-01	559
1959-09-01	463
1959-10-01	407
1959-11-01	362
1959-12-01	405
1960-01-01	417
1960-02-01	391
1960-03-01	419
1960-04-01	461
1960-05-01	472
1960-06-01	535
1960-07-01	622
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

In [103... *###Build various exponential smoothing models on the training data and evaluate the #such as regression,naïve forecast models, simple average models etc. should also b #test data using RMSE.*

```
In [133... train_reg = train.copy()
test_reg = test.copy()

train_reg['t'] = np.arange(len(train_reg))
test_reg['t'] = np.arange(
    len(train_reg),
    len(train_reg)+len(test_reg)
)
ir = LinearRegression()
ir.fit(
    train_reg[['t']],
```

```

train_reg['Pax']
)

forecast_reg = ir.predict(
    test_reg[['t']]
)

from sklearn.metrics import mean_squared_error

rmse_reg = np.sqrt(
    mean_squared_error(
        test_reg['Pax'],
        forecast_reg
    )
)

print("RMSE-" , rmse_reg )

```

RMSE- 71.50326413072636

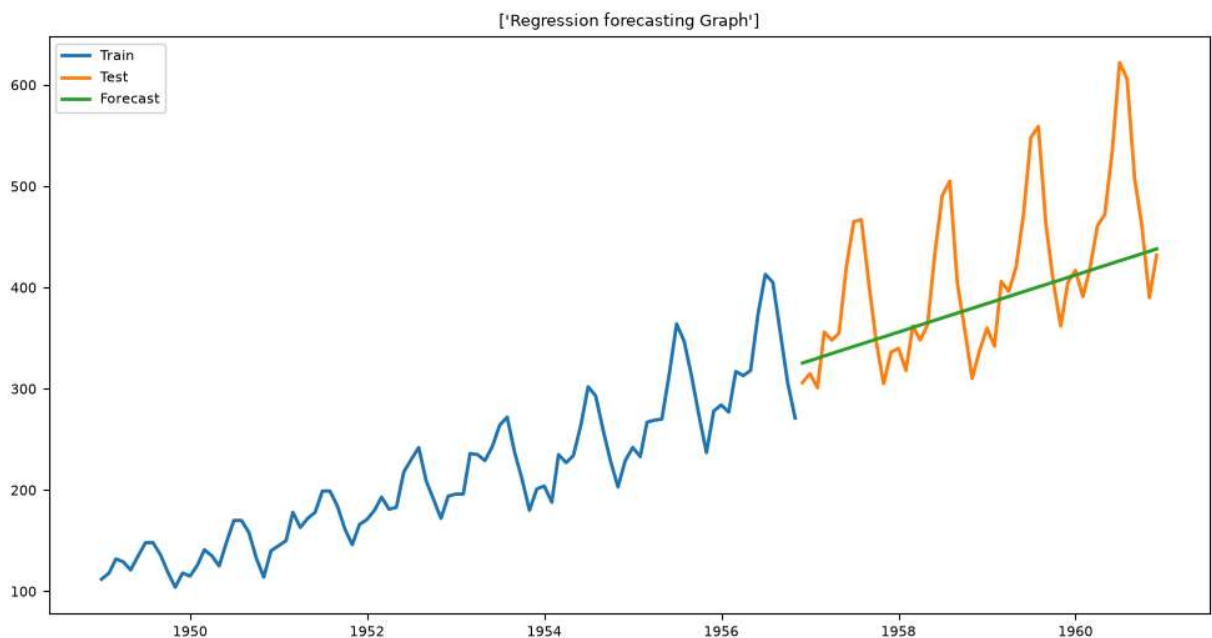
In [134... *##Let's creat output chart against this model*

```

plt.figure(figsize=(12,6))
plt.plot(train.index, train['Pax'])
plt.plot(test.index, test['Pax'])
plt.plot(test.index, forecast_reg)

plt.title(['Regression forecasting Graph'])
plt.legend(["Train", "Test", "Forecast"])
plt.show()

```



In [143... *#Simple Average Model -*

```
In [148... forecast_avg = np.repeat(
                train['Pax'].mean(),
                len(test)
            )

rmse_avg = rmse(
                test['Pax'],
                forecast_avg
            )

print("RMSE:", rmse_avg)
```

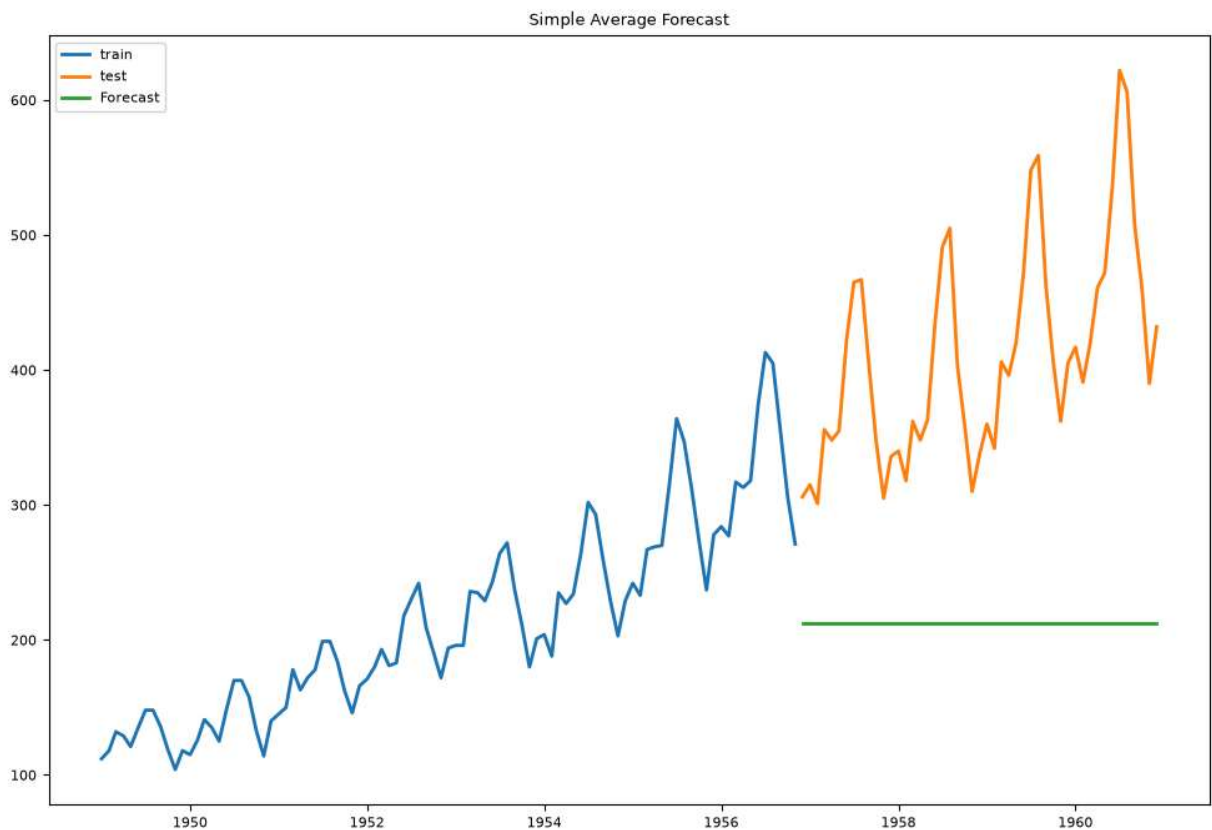
RMSE: 213.45510819738584

```
In [149... ##Let's plot this
```

```
In [157... import matplotlib.pyplot as plt

plt.figure(figsize = (12,8))
plt.plot(train.index, train['Pax'])
plt.plot(test.index, test['Pax'])
plt.plot(test.index, forecast_avg)

plt.title("Simple Average Forecast")
plt.legend(["train", "test", "Forecast"])
plt.show()
```



```
In [158... ##Moving Avg
```

```

In [237... window =12
moving_ma = train['Pax'].tail(window).mean()

Forecast_ma = np.repeat(
    moving_avg,
    len(test)
)

rmse_ma = rmse(
    test['Pax'],
    Forecast_ma
)

print("RMSE-", rmse_ma)

```

RMSE- 115.88486899855299

```

In [163... ##Plot this

```

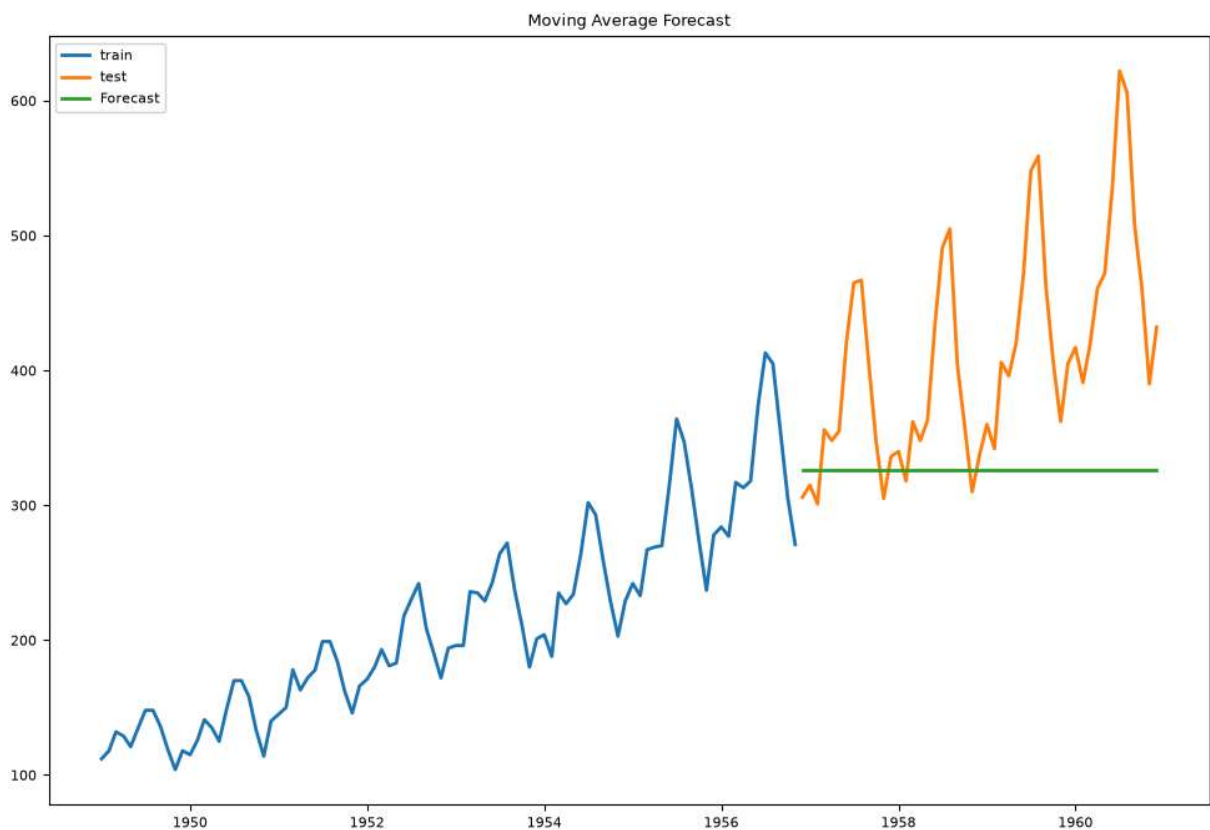
```

In [238... import matplotlib.pyplot as plt

plt.figure(figsize = (12,8))
plt.plot(train.index, train['Pax'])
plt.plot(test.index, test['Pax'])
plt.plot(test.index, Forecast_ma)

plt.title("Moving Average Forecast")
plt.legend(["train", "test", "Forecast"])
plt.show()

```



In []: *##Triple Exponential Smoothing (Holt-Winters) -*

```
In [229... from statsmodels.tsa.holtwinters import ExponentialSmoothing
tes_model = ExponentialSmoothing(
    train['Pax'],
    trend = 'add',
    seasonal = 'mul',
    seasonal_periods = 12
).fit()

tes_forecast = tes_model.forecast(
    len(test)

)

rmse_tse = rmse(
    test['Pax'],
    tes_forecast
)

print('RMSE-',rmse_tse )
```

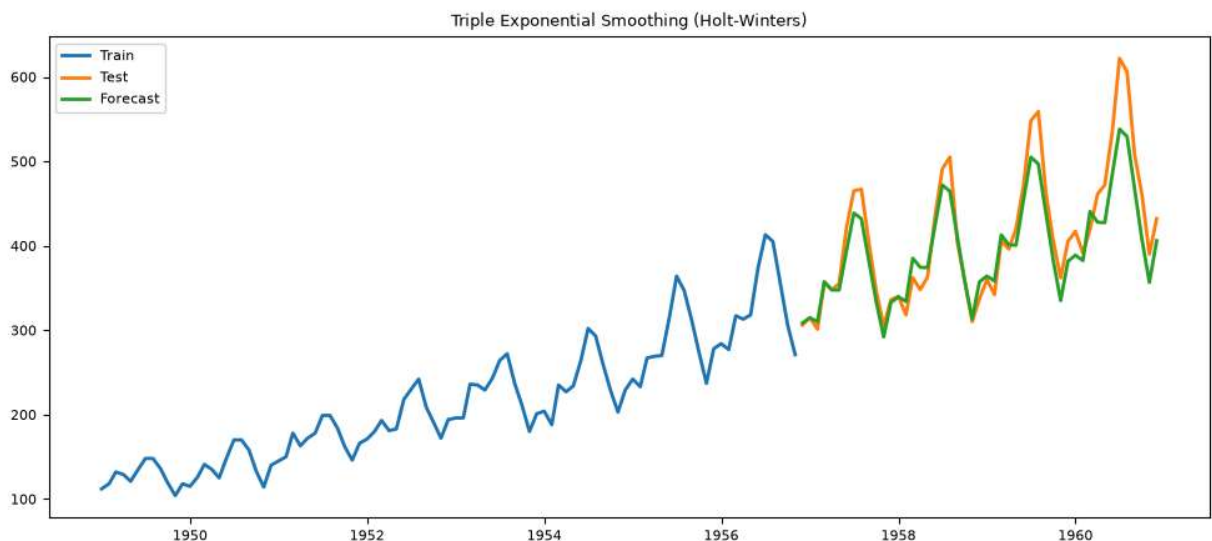
C:\Users\parit\anaconda3_new\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.

self._init_dates(dates, freq)

RMSE- 29.676756347093118

```
In [169... plt.figure(figsize=(12,5))
plt.plot(train.index, train['Pax'])
plt.plot(test.index, test['Pax'])
plt.plot(test.index, tes_forecast)

plt.title('Triple Exponential Smoothing (Holt-Winters)')
plt.legend(['Train', 'Test', 'Forecast'])
plt.show()
```



In [170...] *##Let's plot each RMSE value and check best model*

```
In [187...] results = pd.DataFrame({
    'Model': [
        'Linear Regression',
        'Simple avarage',
        'Moving average',
        'Triple Expo. smoothing'
    ],
    'RMSE': [
        rmse_reg,
        rmse_avg,
        rmse_ma,
        rmse_tse
    ]
})
```

In [188...] *#sort the values*

```
results = results.sort_values(by='RMSE')
print(results)
```

	Model	RMSE
3	Triple Expo. smoothing	29.676756
0	Linear Regression	71.503264
2	Moving average	115.884869
1	Simple avarage	213.455108

In [197...] *##Hence, Triple Exponential Smoothing (Holt-Winters) is the best model to forecast*

In [232...] *# Final model: Triple Exponential Smoothing*
Fit the selected model using complete historical data (Train+Test)

```
tes_final_model = ExponentialSmoothing(
    df1['Pax'],
    trend='add',
    seasonal='mul',
    seasonal_periods=12
).fit()

# Forecast next 12 months

future_forecast = tes_final_model.forecast(12)

print(future_forecast)
```

```
C:\Users\parit\anaconda3_new\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:47
3: ValueWarning: No frequency information was provided, so inferred frequency MS wil
1 be used.
    self._init_dates(dates, freq)
```



```
1961-01-01    445.242402
1961-02-01    418.225352
1961-03-01    465.309711
1961-04-01    494.951206
1961-05-01    505.475846
1961-06-01    573.312580
1961-07-01    663.596282
1961-08-01    654.903928
1961-09-01    546.760946
1961-10-01    488.446880
1961-11-01    415.723589
1961-12-01    460.377961
Freq: MS, dtype: float64
```

In [204... *##Let's plot 12 month future sales value*

```
In [233... plt.figure(figsize=(10, 6))

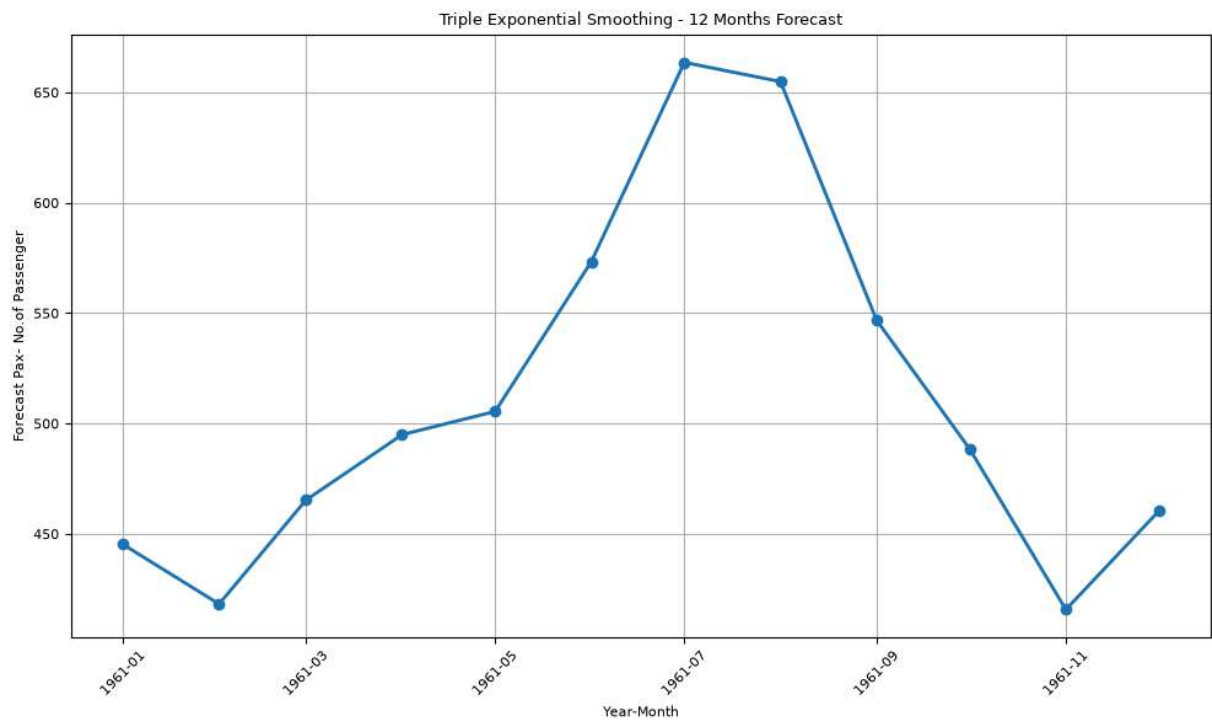
plt.plot(
    future_forecast.index,
    future_forecast.values,
    marker='o'
)

plt.title('Triple Exponential Smoothing - 12 Months Forecast')
plt.xlabel('Year-Month')
plt.ylabel('Forecast Pax- No.of Passenger')

plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()

plt.show()

tes_future_forecast = tes_model.forecast(12)
```



In [239...

```
##This is 12 months demand forecast of 1961 based on the TES model.  
#We can easily find sales of 12 months by using this demand analysis and  
#accordingly we can plan inventory/resources as well.
```